视图层框架设计

在深入讨论 Vue.js 3 各个模块的实现思路和细节之前，先来讨论视图层框架设计方面的内容。为什么呢？这是因为当设计一个框架的时候，框架本身的各个模块之间并不是相互独立的，而是相互关联、相互制约的。因此作为框架设计者，一定要对框架的定位和方向拥有全局的把控，这样才能做好后续的模块设计和拆分。同样，作为学习者，我们在学习框架的时候，也应该从全局的角度对框架的设计拥有清晰的认知，否则很容易被细节困住，看不清全貌。

命令式和声明式

从范式上来看，视图层框架通常分为命令式和声明式，它们各有优缺点。

早年间流行的 jQuery 就是典型的命令式框架。命令式框架的一大特点就是**关注过程**。例如，我们把下面这段话翻译成对应的代码：

```
- 获取 id 为 app 的 div 标签
- 它的文本内容为 hello world
- 为其绑定点击事件
  当点击时弹出提示：ok
```

对应的代码

```
const div = document.querySelector('#app') // 获取 div
div.innerText = 'hello world' // 设置文本内容
div.addEventListener('click', () => { alert('ok') }) // 绑定点击事件
```

可以看到，自然语言描述能够与代码产生一一对应的关系，代码本身描述的是“做事的过程”，这符合我们的逻辑直觉

与命令式框架更加关注过程不同，声明式框架更加**关注结果**。结合 Vue.js，我们来看看如何实现上面自然语言描述的功能：

```
<div id='app' @click="() => alert('ok')">hello world</div>
```

性能与可维护性的权衡

命令式和声明式各有优缺点，在框架设计方面，则体现在性能与可维护性之间的权衡。这里我们先抛出一个结论：**声明式代码的性能不优于命令式代码的性能**。

还是拿上面的例子来说，假设现在我们要将 `div` 标签的文本内容修改为 `hello vue3`，那么如何用命令式代码实现呢？很简单，因为我们明确知道要修改的是什么，所以直接调用相关命令操作即可：

```
div.textContent = 'hello vue3' // 直接修改
```

现在思考一下，还有没有其他办法比上面这句代码的性能更好？答案是“没有”。可以看到，理论上命令式代码可以做到极致的性能优化，因为我们明确知道哪些发生了变更，只做必要的修改就行了。但是声明式代码不一定能做到这一点，因为它描述的是结果：

```
// 之前:
<div @click="() => alert('ok')">hello world</div>
// 之后:
<div @click="() => alert('ok')">hello vue3</div>
```

对于框架来说，为了实现最优的更新性能，它需要找到前后的差异并只更新变化的地方，但是最终完成这次更新的代码仍然是：

```
div.textContent = 'hello vue3' // 直接修改
```

如果我们把直接修改的性能消耗定义为 A，把找出差异的性能消耗定义为 B，那么有：

- 命令式代码的更新性能消耗 = A
- 声明式代码的更新性能消耗 = B + A

这就体现了我们在框架设计上要做出的关于可维护性与性能之间的权衡。在采用声明式提升可维护性的同时，性能就会有一定的损失，而框架设计者要做的就是：**在保持可维护性的同时让性能损失最小化**。

虚拟 DOM

真实 DOM 和虚拟 DOM

Real DOM，真实 DOM， 意思为文档对象模型，是一个结构化文本的抽象，在页面渲染出的每一个结点都是一个真实 DOM 结构，如下：

```
<div id='root'>
  <h1>Hello World</h1>
</div>
```

Virtual Dom，虚拟 DOM 本质上是以 JavaScript 对象形式存在的对 DOM 的描述

它们的区别:

- 虚拟 DOM 不会进行排版与重绘操作，而真实 DOM 会频繁重排与重绘
- 虚拟 DOM 的总损耗是“虚拟 DOM 增删改+真实 DOM 差异增删改+排版与重绘”，真实 DOM 的总损耗是“真实DOM完全增删改+排版与重绘”

它们的优缺点:

真实 DOM 的优势：

- 易用

缺点：

- 效率低，解析速度慢，内存占用量过高
- 性能差：频繁操作真实 DOM，易于导致重绘与回流

使用虚拟 DOM 的优势如下：

- 简单方便：如果使用手动操作真实 DOM 来完成页面，繁琐又容易出错，在大规模应用下维护起来也很困难
- 性能方面：使用 Virtual DOM，能够有效避免真实 DOM 数频繁更新，减少多次引起重绘与回流，提高性能
- 跨平台：React 借助虚拟 DOM， 带来了跨平台的能力，一套代码多端运行

缺点：

- 在一些性能要求极高的应用中虚拟 DOM 无法进行针对性的极致优化
- 首次渲染大量 DOM 时，由于多了一层虚拟 DOM 的计算，速度比正常稍慢

运行时和编译时

当设计一个框架的时候，我们有三种选择：纯运行时的、运行时 + 编译时的或纯编译时的。

先聊聊纯运行时的框架。假设我们设计了一个框架，它提供一个 `Render` 函数，用户可以为该函数提供一个树型结构的数据对象，然后 `Render` 函数会根据该对象递归地将数据渲染成 DOM 元素。我们规定树型结构的数据对象如下：

```
const obj = {
  tag: 'div',
  children: [
```

```
    { tag: 'span', children: 'hello world' }
  ]
}
```

每个对象都有两个属性：`tag` 代表标签名称，`children` 既可以是一个数组（代表子节点），也可以直接是一段文本（代表文本子节点）。接着，我们来实现 `Render` 函数：

```
function Render(obj, root) {
  const el = document.createElement(obj.tag)
  if (typeof obj.children === 'string') {
    const text = document.createTextNode(obj.children)
    el.appendChild(text)
  } else if (obj.children) {
    // 数组，递归调用 Render，使用 el 作为 root 参数
    obj.children.forEach((child) => Render(child, el))
  }

  // 将元素添加到 root
  root.appendChild(el)
}
```

现在我们回过头来思考一下用户是如何使用 `Render` 函数的。可以发现，用户在使用它渲染内容时，直接为 `Render` 函数提供了一个树型结构的数据对象。这里面不涉及任何额外的步骤，用户也不需要学习额外的知识。但是有一天，你的用户抱怨说：“手写树型结构的数据对象太麻烦了，而且不直观，能不能支持用类似于 HTML 标签的方式描述树型结构的数据对象呢？”你看了看现在的 `Render` 函数，然后回答：“抱歉，暂不支持。”实际上，我们刚刚编写的框架就是一个**纯运行时**的框架。

为了满足用户的需求，你开始思考，能不能引入编译的手段，把 HTML 标签编译成树型结构的数据对象，这样不就可以继续使用 `Render` 函数了吗？

```
const html = `
<div>
  <span>hello world</span>
</div>
`

// 调用 Compiler 编译得到树型结构的数据对象
const obj = Compiler(html)
// 再调用 Render 进行渲染
Render(obj, document.body)
```

上面这段代码能够很好地工作，这时我们的框架就变成了一个**运行时 + 编译时**的框架。它既支持运行时，用户可以直接提供数据对象从而无须编译；又支持编译时，用户可以提供 HTML 字符串，我们将其编译为数据对象后再交给运行时处理。

我们用简单的例子讲解了框架设计层面的**运行时**、**编译时**以及**运行时 + 编译时**。我们发现，一个框架既可以是纯运行时的，也可以是纯编译时的，还可以是既支持运行时又支持编译时的。那么，它们都有哪些优缺点呢？是不是既支持运行时又支持编译时的框架最好呢？为了搞清楚这个问题，我们逐个分析。

首先是纯运行时的框架。由于它没有编译的过程，因此我们没办法分析用户提供的内容，但是如果加入编译步骤，可能就大不一样了，我们可以分析用户提供的内容，看看哪些内容未来可能会改变，哪些内容永远不会改变，这样我们就可以在编译的时候提取这些信息，然后将其传递给 `Render` 函数，`Render` 函数得到这些信息之后，就可以做进一步的优化了。然而，假如我们设计的框架是纯编译时的，那么它也可以分析用户提供的内容。由于不需要任何运行时，而是直接编译成可执行的 JavaScript 代码，因此性能可能会更好，但是这种做法有损灵活性，即用户提供的内容必须编译后才能用。实际上，在这三个方向上业内都有探索，其中 `Svelte` 就是纯编译时的框架，但是它的真实性能可能达不到理论高度。Vue.js 3 仍然保持了运行时 + 编译时的架构，在保持灵活性的基础上能够尽可能地去优化。等到后面讲解 Vue.js 3 的编译优化相关内容时，你会看到 Vue.js 3 在保留运行时的情况下，其性能甚至不输纯编译时的框架。

参考文档:

面试官：说说 Real DOM和 Virtual DOM 的区别？优缺点？

[react] 什么是虚拟dom？虚拟dom比操作原生dom要快吗？虚拟dom是如何转变成真实dom并渲染到页面的？